

4

Lenguajes de 4ª generación: Oracle PL/SQL

Mercedes García

Virginia Francisco Gilmartín

Fernando Sáenz Pérez

Facultad de Informática

Universidad Complutense



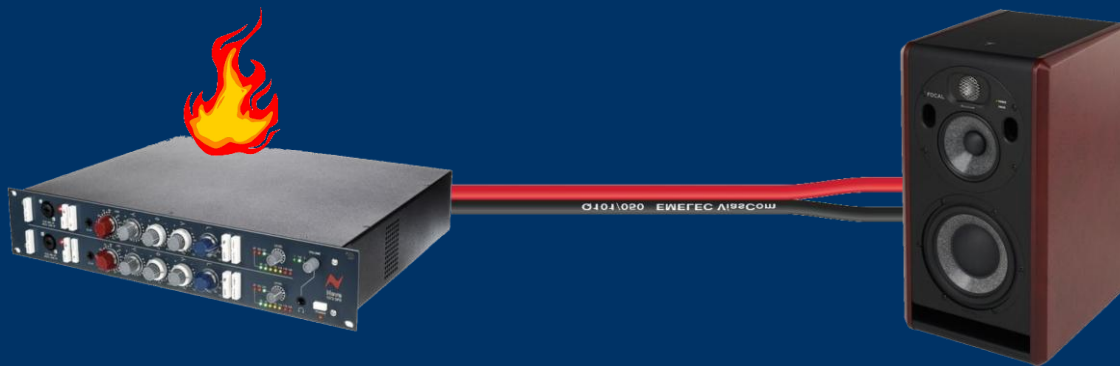
Índice

- ❑ *Introducción*
- ❑ *Bloques*
 - ❑ Variables
 - ❑ Salida de texto
 - ❑ Funciones embebidas
 - ❑ Instrucciones SQL embebidas
 - ❑ Condicionales
 - ❑ Bucles
 - ❑ Cursores
 - ❑ Registros
 - ❑ Excepciones
- ❑ *Procedimientos*
- ❑ *Funciones*



Introducción

- ✓ Los sistemas gestores de bases de datos incorporan un lenguaje imperativo que amplía el lenguaje SQL con elementos de la programación estructurada
- ✓ Permiten utilizar condiciones y bucles al estilo de otros lenguajes como Java, C++, etc.
- ✓ Evitan el **desajuste de impedancia**:
 - Usan los mismos tipos que SQL
 - Añaden estructuras de datos para representar a las relaciones



Introducción

- ✓ Se usan para:
 - Realizar consultas avanzadas
 - Tareas que no se pueden realizar solo con SQL o para las que SQL no resulta adecuado
 - Realizar tareas administrativas sobre la base de datos: copia de valores antiguos, auditorías, control de usuarios, etc...
- ✓ Los programas en este lenguaje imperativo pueden almacenarse en la propia base de datos o en archivos externos



Introducción

- ✓ Cada SGBD (no todos) aporta su propio lenguaje de cuarta generación:
 - Access: *No*
 - DES: Scripting
 - MySQL: Procedure Language (limitado)
 - DB2: SQL-PL
 - SQL Server: Transact-SQL
 - PostgreSQL: PL/pgSQL
 - Oracle: PL/SQL

Generaciones de los lenguajes de programación:

- **1GL** (First Generation Language): código máquina
- **2GL**: ensambladores
- **3GL**: abstraen parte de la arquitectura (C, Java...)
- **4GL**: incorporan mecanismos declarativos (PL/SQL)
- **5GL**: lenguajes declarativos de propósito general (Prolog, Lisp...)



Bloque

- ✓ Fragmento de código que puede ser interpretado por Oracle

```
DECLARE  
    variables
```

← **Declaraciones (opcional)**

```
BEGIN  
    sentencias PL/SQL
```

← **Sentencias ejecutables**

```
EXCEPTION  
    excepciones
```

← **Tratamiento de excepciones (opcional)**

```
END;
```

← **Final del bloque**



Ejemplo de bloque

- ✓ Bloque para 'Hello, World!'

```
SET SERVEROUTPUT ON SIZE 1000000;
```

```
BEGIN
```

```
    dbms_output.PUT_LINE('Hello, World!');
```

```
END;
```



Variables. Declaración

- ✓ Las variables se declaran dentro de la sentencia `DECLARE`
- ✓ Sintaxis:

```
nombreVar [CONSTANT] tipoVar [NOT NULL]  
          [[:= | DEFAULT] expresión];
```

- ✓ Reglas para los nombres de variable:
 - Dos variables distintas pueden tener el mismo nombre si están en bloques diferentes
 - El nombre de una variable no debería de ser el mismo que el de una columna de una tabla usada en el bloque



Variables. Consideraciones

- ✓ Las variables se inicializan por defecto al valor `NULL`
- ✓ Cualquier variable declarada como `NOT NULL` tiene que ser inicializada con un valor
- ✓ La inicialización (con `:=` o `DEFAULT`, que son sinónimos) puede incluir cualquier expresión legal de PL/SQL que devuelva un valor del tipo de la variable que se está declarando
- ✓ No se pueden declarar varias variables en la misma instrucción
- ✓ Las variables declaradas en un bloque concreto se eliminan cuando este acaba



Variables. Tipos

CHAR (n)	Texto de anchura fija
VARCHAR (n)	Texto de anchura variable
NUMBER [(p [, s])]	Número. Opcionalmente puede indicar la cantidad total de dígitos (p: precision) y, de ellos, la cantidad que son decimales (s: scale)
DATE	Almacena fechas
INTEGER	Enteros equivalente a NUMBER (38)
BOOLEAN	Permite almacenar los valores TRUE (verdadero) y FALSE (falso)



Variables. Declaración. Ejemplo

```
DECLARE
  v_numero  NUMBER(3);
  v_fecha   DATE;
  PI  CONSTANT  NUMBER(9,7) := acos(-1);

BEGIN
  ...
END;
```

¡Operador de asignación!
(como en Pascal)



Variables. La expresión %TYPE

- ✓ La expresión %TYPE se usa para dar a una variable el mismo tipo de otra variable o el tipo de una columna de una tabla de la base de datos
- ✓ Ejemplo:

```
DECLARE  
    v_nombre      customers.name%TYPE;  
    v_precio      NUMBER(7,1);  
    v_precioMin   v_precio%TYPE;
```

- La variable v_nombre tendrá el mismo tipo que la columna name de la tabla customers
- La variable v_precioMin tendrá el tipo de la variable v_precio



Salida de texto

- ✓ Los programas que se ejecutan en la base de datos tienen una salida de texto (consola) donde pueden escribir mediante el paquete `DBMS_OUTPUT`:
 - `PUT_LINE (...)` y `PUT (...)` → funciones para mostrar datos (un argumento de cualquier tipo)
 - `NEW_LINE ()` → función para escribir una línea en blanco (sin argumentos, pero con paréntesis)



Entrada de texto

- ✓ Ehhhh.... ¡pues no existe! ¡No hay un `DBMS_INPUT`!
- ✓ ¿Por qué?
 - Los usuarios/aplicaciones que emiten una transacción no tienen permiso para inyectar datos en el contexto de un bloque; solo pueden recibirlos en la consola de Oracle
 - Solo se pueden leer datos en scripts (y con limitaciones)



Comentarios

- ✓ En un bloque PL/SQL se admiten dos tipos de comentarios:
 - Comentarios de una línea. Deben comenzar por '--'. Es posible añadir un comentario de línea al final de una instrucción
 - Comentarios de más de una línea. El comentario debe incluirse dentro de '/*' y '*/' (como en C, Java...)



Bloque. Ejemplo

```
DECLARE
    parámetro CONSTANT NUMBER(3) := 123;
BEGIN
    /* Esto es un comentario
       que ocupa dos líneas */
    IF parámetro = 123 THEN -- esto es otro comentario
        parámetro := parámetro * 2
        DBMS_OUTPUT.PUT('El valor es: ');
        DBMS_OUTPUT.PUT_LINE(parámetro);
    ELSE
        DBMS_OUTPUT.PUT_LINE('Hay un error. ');
    END IF;
END;
```

¡Operador de comparación!
(como en SQL)



Cursores implícitos. SELECT INTO

- ✓ Un **cursor implícito** o **en línea** se utiliza para almacenar lo que retorna una instrucción **SELECT** en variables definidas por el usuario
- ✓ Se produce error si la **SELECT** devuelve más de una fila o si no devuelve ninguna fila
- ✓ Ejemplo:

```
DECLARE
    v_salario NUMBER(9,2);
    v_nombre VARCHAR2(50);
BEGIN
    SELECT    salario, nombre
      INTO v_salario, v_nombre
    FROM empleados
   WHERE cod_empleado = 12344;

    DBMS_OUTPUT.PUT('El nuevo salario de ');
    DBMS_OUTPUT.PUT(v_nombre || ' serán ');
    DBMS_OUTPUT.PUT_LINE(v_salario*1.2 || ' €.');
END;
```



Instrucciones SQL embebidas. Insert, Update y Delete (I)

- ✓ Se pueden usar con variables
- ✓ Ejemplo:

```
DECLARE
    v_salario empleados.salario%TYPE;
BEGIN
    SELECT salario
    INTO v_salario
    FROM empleados
    WHERE cod_empleado = 12344;

    UPDATE empleados
    SET salario = v_salario*1.2
    WHERE cod_empleado = 12344;
END;
```

¡Y esto es una asignación en SQL!



Instrucciones SQL embebidas. Insert, Update y Delete (II)

- ✓ Es posible también insertar los datos recuperados mediante una consulta `SELECT` (ya lo sabíamos)
- ✓ Ejemplo:

```
INSERT INTO Préstamo  
SELECT * FROM NuevosPréstamos;
```

- ✓ Es posible eliminar/actualizar mediante subconsultas (esto también lo sabíamos)
- ✓ Ejemplo:

```
DELETE FROM Clientes  
WHERE Clientes.NumPréstamo NOT IN (SELECT NumPréstamo  
                                   FROM Préstamo);  
  
UPDATE Préstamo  
SET Sucursal = 'Centro'  
WHERE Sucursal IN (SELECT sucursal  
                   FROM SucursalesCerradas);
```



Sentencias condicionales. IF

✓ Sintaxis:

```
IF condicion1 THEN
    instrucciones
END IF;
```

✓ Ejemplo:

```
IF salario > 3000 THEN
    DBMS_OUTPUT.PUT_LINE('Muy buen empleo');
END IF;
```



Sentencias condicionales. IF-THEN-ELSE

✓ Sintaxis:

```
IF condición THEN
    instrucciones
ELSE
    instrucciones
END IF;
```

✓ Ejemplo:

```
IF salario > 3000 THEN
    DBMS_OUTPUT.PUT_LINE('Muy buen empleo');
ELSE
    DBMS_OUTPUT.PUT_LINE('Salario mejorable');
END IF;
```



Sentencias condicionales. IF-THEN-ELSIF

✓ Sintaxis:

```
IF condición1 THEN
    instrucciones
ELSIF condición2 THEN
    instrucciones
[ELSIF ...]
[ELSE
    instruccionesElse]
END IF;
```

✓ Ejemplo:

```
IF salario > 3000 THEN
    DBMS_OUTPUT.PUT_LINE('Muy buen empleo');
ELSIF salario > 2000 THEN
    DBMS_OUTPUT.PUT_LINE('No está del todo mal');
ELSE
    DBMS_OUTPUT.PUT_LINE('Salario bajo');
END IF;
```



Sentencias condicionales. CASE

- ✓ La sentencia `CASE` devuelve un resultado tras evaluar una expresión
- ✓ Sintaxis propia de SQL:

```
CASE selector
  WHEN expresión1 THEN resultado1
  WHEN expresión2 THEN resultado2
  ...
  [ELSE resultadoElse]
END;
```

- ✓ Ejemplo:

```
v_valor := CASE estudios
            WHEN 'A' THEN 'Nivel superior'
            WHEN 'B' THEN 'Nivel grado medio'
            WHEN 'C' THEN 'Nivel básico'
            WHEN 'D' THEN 'Sin estudios'
            ELSE 'Desconocido'
            END;
```



Bucles (I)

- ✓ Los bucles se pueden anidar
- ✓ Tres tipos de bucles:
 - **LOOP**: Se repiten las instrucciones del bucle mientras la condición sea falsa

```
LOOP
    instrucciones
    EXIT [WHEN <condición>]
END LOOP;
```

- **WHILE**: Se repiten las instrucciones del bucle mientras la condición sea cierta

```
WHILE <condición>
    LOOP
        instrucciones
    END LOOP;
```



Bucles (II)

— FOR:

- ✓ Se utiliza una variable contadora que no tiene por qué estar declarada en el `DECLARE`. Esta variable se declara automáticamente en el propio `FOR` y se elimina cuando este finaliza
- ✓ Se indica el valor inicial de la variable contador y el valor final (el incremento irá de uno en uno)
- ✓ Si se utiliza la cláusula `REVERSE`, entonces el contador cuenta desde el valor final al inicial restando 1
- ✓ ¿No tenemos `STEP`? Pues no... (¡hasta Basic lo tenía!)

```
FOR v_contador IN [REVERSE] valorIni..valorFin  
LOOP  
    Instrucciones  
END LOOP;
```



Recorrido simple de cursores implícitos

- ✓ Puedes usar un bucle `FOR` para recorrer un cursor implícito:

```
FOR v_filas IN (SELECT ...) LOOP
    ... Instrucciones ...
END LOOP;
```

- ✓ El acceso a cada una de las columnas del resultado de la `SELECT` es igual que en un registro (se verán más tarde):
 - `v_filas.nombre_columna`
- ✓ Inconveniente: no se puede acceder al atributo `%TYPE`.
- ✓ Ventaja: no es necesario declarar variables para recoger los datos (como `v_filas`).
- ✓ Ejemplo:

```
FOR v_filas IN (SELECT * FROM programadores) LOOP
    DBMS_OUTPUT.PUT_LINE(v_filas.dni||'-'||v_filas.nombre);
END LOOP;
```



Cursores explícitos

- ✓ Cuando una consulta `SELECT` devuelve más de una tupla se pueden utilizar cursores explícitos para acceder a cada una de las tuplas devueltas
- ✓ Sintaxis:

```
DECLARE
    ...
    CURSOR nombre IS sentencia_select;
    ...
BEGIN
    ...
END;
```

- ✓ El cursor posee un puntero señalando a cada una de las filas del cursor
- ✓ Para recorrer las filas del cursor hacemos avanzar el puntero
- ✓ Cuando un cursor está abierto y los datos referenciados por la `SELECT` cambian, estos cambios no se actualizan en el cursor



Apertura del cursor

`OPEN nombre_cursor`

- ✓ Reserva memoria suficiente para el cursor
- ✓ Ejecuta la sentencia `SELECT` a la que se refiere el cursor
- ✓ Coloca el puntero de recorrido de registros al principio del conjunto de resultados
- ✓ Si la sentencia `SELECT` del cursor no devuelve registros, Oracle no devuelve ninguna excepción. Hasta intentar leer del cursor no sabremos si hay resultados o no
- ✓ Ejemplo:

`OPEN c_provincias;`



Cursores. Instrucción FETCH

```
FETCH nombre_cursor INTO lista_variables
```

- ✓ Esta instrucción almacena el contenido de la fila a la que apunta actualmente el puntero de la lista de variables indicada
- ✓ La lista de variables debe tener el mismo tipo y número que las columnas representadas en él
- ✓ Tras esta instrucción el puntero de registros avanza a la siguiente fila (si la hay)
- ✓ Ejemplo:

```
FETCH c_provincias INTO (v_nombre, v_población);
```



Cierre del cursor

```
CLOSE nombre_cursor;
```

- ✓ Al cerrar el cursor se libera la memoria que ocupa y se impide su procesamiento
- ✓ Tras cerrar el cursor se podría abrir de nuevo
- ✓ Ejemplo:

```
CLOSE c_provincias;
```



Cursores. Atributos

<code>%NOTFOUND</code>	Devuelve verdadero si la última instrucción <code>FETCH</code> no devolvió ningún valor
<code>%FOUND</code>	Devuelve verdadero si la última instrucción <code>FETCH</code> devolvió una fila
<code>%ROWCOUNT</code>	Devuelve el número de filas que se han recorrido en el cursor. Inicialmente vale cero
<code>%ISOPEN</code>	Devuelve verdadero si el cursor está abierto



Cursores. Procesamiento

- ✓ Es necesario realizar las siguientes operaciones para operar con un cursor:

1. Abrir el cursor con:

`OPEN <nombreCursor>`

2. Procesar el cursor con:

`FETCH <nombre cursor> INTO <lista variables>`

- La instrucción `FETCH` permite recorrer el cursor fila a fila hasta que el puntero llegue al final (hasta que el cursor esté vacío)

3. Cerrar el cursor con:

`CLOSE <nombre cursor>`



Cursores. Ejemplo

```
DECLARE
    v_nombre localidades.nombre%TYPE;
    v_total localidades.población%TYPE;
    CURSOR c_provincias IS
        SELECT P.nombre, SUM(L.población) AS total_población
        FROM localidades L INNER JOIN provincias P
            ON L.provincia = P.nombre
        GROUP BY P.nombre;

BEGIN
    OPEN c_provincias ;
    -- 1. Abrimos el cursor
    LOOP
        -- 2. Procesamos el cursor
        FETCH c_provincias
        INTO v_nombre, v_total;

        /*Ejecutamos el bucle hasta que recorramos todas las filas del
        cursor*/
        EXIT WHEN c_provincias%NOTFOUND;

        INSERT INTO listado VALUES (v_nombre, v_total);

        DBMS_OUTPUT.PUT_LINE('La tupla se ha insertado');
    END LOOP;
    -- 3. Cerramos el cursor
    CLOSE c_provincias;
END;
```



Cursores paramétricos

- ✓ Al igual que los cursores implícitos, los cursores explícitos también se pueden conectar con variables o constantes de PL/SQL
- ✓ En su declaración se añaden los parámetros y al abrirlo se le pasan los valores:

```
CURSOR <nombre> (<parámetro1> <tipo1>,  
                ..., (<parámetron> <tipon>)  
IS <sentencia_select;>
```

- ✓ Esta sentencia SELECT contendrá los nombres de los parámetros.
- ✓ Al abrir un cursor de este tipo se le pasarán los valores entre paréntesis:

```
OPEN <nombre> (valor1, ..., valorn )
```



Cursores paramétricos. Ejemplo

✓ Declaración:

```
CURSOR c_EmpDept (p_Dept Empleados.Dept%TYPE)
IS SELECT DNI, Nombre, Sueldo
FROM Empleados
WHERE Dept=p_Dept;
```

✓ Apertura:

```
OPEN c_EmpDept ('Recursos humanos');
```



Recorrido simple de cursores explícitos

- ✓ Al igual que con los implícitos, puedes usar un bucle FOR para recorrer el cursor explícito:

```
FOR variable IN nombre_cursor LOOP  
    ... Instrucciones ...  
END LOOP;
```

- ✓ Inconveniente: no se puede acceder a las propiedades del cursor (%NOTFOUND, %ROWCOUNT, %ISOPEN...)
- ✓ Ventajas:
 - Acceso simplificado (no necesita OPEN, CLOSE, FETCH).
 - No necesita declarar variables para recoger los datos.



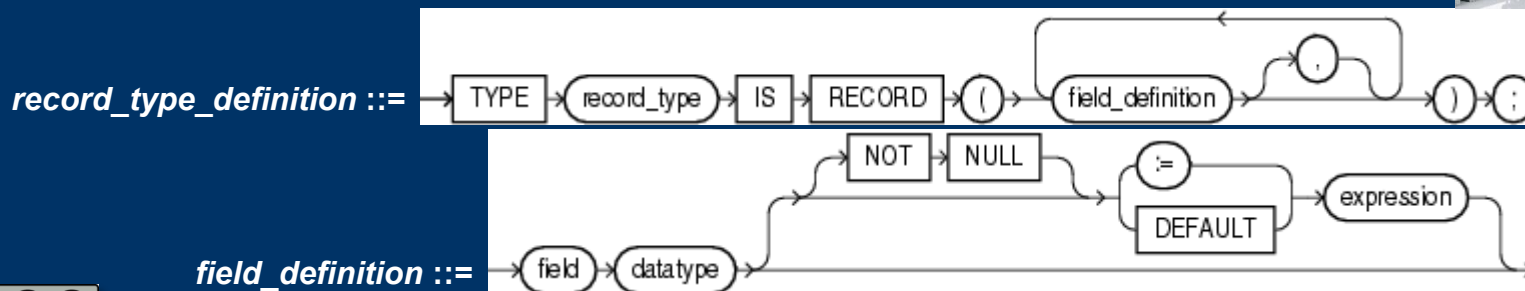
Registros (I)

- ✓ Un registro es una estructura de datos
- ✓ Mediante declaraciones de tipos de datos se pueden definir estos registros
- ✓ Sintaxis simple:

El concepto de tipo de datos en Oracle es más cercano al de clase de objetos: también se pueden definir métodos y funciones, aunque no lo veremos en esta asignatura

```
TYPE t_nombreReg IS RECORD (  
    campo1 tipo1 [[NOT] NULL],  
    ...,  
    campoN tipoN [[NOT] NULL]);
```

- ✓ Diagrama de ferrocarril (*railroad*):



Registros. Ejemplo

```
-- Declaramos el tipo r_persona como registro
TYPE r_persona IS RECORD(
    nombre          VARCHAR(25),
    apellido1       VARCHAR(25),
    apellido2       VARCHAR(25),
    salario         NUMBER(10) );

-- Declaramos una variable de tipo r_persona
v_cliente_1  r_persona;

/* Asignamos valores a cada uno de los campos
   del registro */
v_cliente_1.nombre      := 'Ana';
v_cliente_1.salario     := 30000;
```



Registros (II)

- ✓ Para asignar a un registro la estructura de una tabla o cursor utilizamos `%ROWTYPE`

```
DECLARE
    r_empleados    empleados%ROWTYPE;      -- Tabla
    r_provincias   c_provincias%ROWTYPE;    -- Cursor
```

- `r_empleados` es un registro que constará de los mismos campos y tipos que las columnas de la tabla `empleados`
- `r_provincias` es un registro que constará de los mismos campos y tipos que las filas del cursor `c_provincias`



Recorrido de cursores con registros

- ✓ La forma más habitual de recorrer cursores es con registros:
 1. Abrir un cursor antes de comenzar el bucle `FOR` / `LOOP` / `WHILE`
 2. Recorrer todas las filas del cursor y almacenar el contenido de cada fila en una variable de tipo registro
 - La variable de tipo registro utilizada en el bucle se crea al inicio del bucle y se elimina cuando este finaliza
 3. Cerrar el cursor cuando finalice el bucle



Recorrido de cursores con registros. Ejemplo

```
DECLARE
    CURSOR c_provincias IS
        SELECT P.nombre, SUM(L.población) AS población_total
        FROM localidades L INNER JOIN provincias P
            ON L.provincia = P.nombre
        GROUP BY P.nombre;
    TYPE r_provincias IS RECORD (
        nombre          localidades.nombre%TYPE,
        población_total localidades.población%TYPE);
    v_provincias r_provincias;
BEGIN
    OPEN c_provincias;
    LOOP
        FETCH c_provincias INTO v_provincias;
        EXIT WHEN c_provincias%NOTFOUND;
        DBMS_OUTPUT.PUT(v_provincias.nombre || ',');
        DBMS_OUTPUT.PUT_LINE(v_provincias.población);
    END LOOP;
    CLOSE c_provincias;
END;
```



Recorrido de cursores con registros. Ejemplo

```
DECLARE
    CURSOR c_provincias IS
        SELECT P.nombre, SUM(L.población) AS población_total
        FROM localidades L INNER JOIN provincias P
            ON L.provincia = P.nombre
        GROUP BY P.nombre;
```

¡Mejor así!

```
    v_provincias c_provincias%TYPE;
BEGIN
    OPEN c_provincias;
    LOOP
        FETCH c_provincias INTO v_provincias;
        EXIT WHEN c_provincias%NOTFOUND;
        DBMS_OUTPUT.PUT(v_provincias.nombre || ',');
        DBMS_OUTPUT.PUT_LINE(v_provincias.población);
    END LOOP;
    CLOSE c_provincias;
END;
```



Recorrido de cursores con registros. Ejemplo

```
DECLARE
```

```
    CURSOR c_provincias IS
```

```
        SELECT P.nombre, SUM(L.población) AS población_total
```

```
        FROM localidades L INNER JOIN provincias P
```

```
            ON L.provincia = P.nombre
```

```
        GROUP BY P.nombre;
```

¡Mejor aún así!

```
BEGIN
```

```
    FOR v_provincia IN c_provincias
```

```
    LOOP
```

```
        DBMS_OUTPUT.PUT(v_provincia.nombre || ',');
```

```
        DBMS_OUTPUT.PUT_LINE(v_provincia.población);
```

```
    END LOOP;
```

```
END;
```

¿Podrías simplificarlo aún más?



Excepciones

- ✓ Una excepción es un evento que causa que la ejecución de un programa finalice (si no se captura)
- ✓ Las excepciones pueden producirse por un error detectado automáticamente por Oracle o pueden ser provocadas manualmente en el programa por el desarrollador
- ✓ Las excepciones se pueden capturar a fin de que el programa controle la situación excepcional (claro, de aquí viene el nombre...)
- ✓ La captura se realiza en el contexto delimitado entre `EXCEPTION` y el `END` del bloque



Excepciones. Sintaxis

```
DECLARE
    sección_declaraciones
BEGIN
    instrucciones
EXCEPTION
    WHEN excepción1 [OR excepción2 ...] THEN
        instrucciones
    [WHEN excepción3 [OR...] THEN instrucciones]
    [WHEN OTHERS THEN instrucciones]
END;
```



Excepciones predefinidas (algunas)

Exception	Raised when
<code>NO_DATA_FOUND</code>	A <code>SELECT INTO</code> statement returns no rows, or your program references a deleted element in a nested table or an uninitialized element in an index-by table. SQL aggregate functions such as <code>AVG</code> and <code>SUM</code> always return a value or a null. So, a <code>SELECT INTO</code> statement that calls an aggregate function never raises <code>NO_DATA_FOUND</code> . The <code>FETCH</code> statement is expected to return no rows eventually, so when that happens, no exception is raised.
<code>ROWTYPE_MISMATCH</code>	The host cursor variable and PL/SQL cursor variable involved in an assignment have incompatible return types. For example, when an open host cursor variable is passed to a stored subprogram, the return types of the actual and formal parameters must be compatible.
<code>TIMEOUT_ON_RESOURCE</code>	A time-out occurs while Oracle is waiting for a resource.
<code>TOO_MANY_ROWS</code>	A <code>SELECT INTO</code> statement returns more than one row.
<code>VALUE_ERROR</code>	An arithmetic, conversion, truncation, or size-constraint error occurs. For example, when your program selects a column value into a character variable, if the value is longer than the declared length of the variable, PL/SQL aborts the assignment and raises <code>VALUE_ERROR</code> . In procedural statements, <code>VALUE_ERROR</code> is raised if the conversion of a character string into a number fails. (In SQL statements, <code>INVALID_NUMBER</code> is raised.)
<code>ZERO_DIVIDE</code>	Your program attempts to divide a number by zero.



Excepciones. Ejemplo

```
DECLARE
    v_ratio NUMBER(3,1);
BEGIN
    SELECT precio/ganancia
    INTO v_ratio
    FROM stocks
    WHERE id = 'XYZ';

    INSERT INTO stats(id, ratio)
    VALUES ('XYZ', v_ratio);

EXCEPTION
    WHEN ZERO_DIVIDE THEN
        INSERT INTO stats(id, ratio)
        VALUES ('XYZ', NULL);

END;
```

Nombre de excepción
predeterminada



Excepciones de usuario

- ✓ PL/SQL permite al usuario definir sus propias excepciones
- ✓ Las excepciones deben ser declaradas en el segmento `DECLARE` de un bloque

```
DECLARE  
    declaraciones ...  
    mi_excepción EXCEPTION;
```

- ✓ Se lanzan de forma explícita utilizando la sentencia `RAISE`



Excepciones de usuario. Ejemplo

```
DECLARE
    v_unidades NUMBER;
    v_margen    NUMBER;
    e_sin_stock EXCEPTION;
BEGIN
    SELECT stock
    INTO v_unidades
    FROM productos
    WHERE código = '123';

    IF v_unidades < v_margen THEN
        RAISE e_sin_stock;
    END IF;
EXCEPTION
    WHEN e_sin_stock THEN
        DBMS_OUTPUT.PUT_LINE('No hay suficientes unidades.');
```

```
END;
```

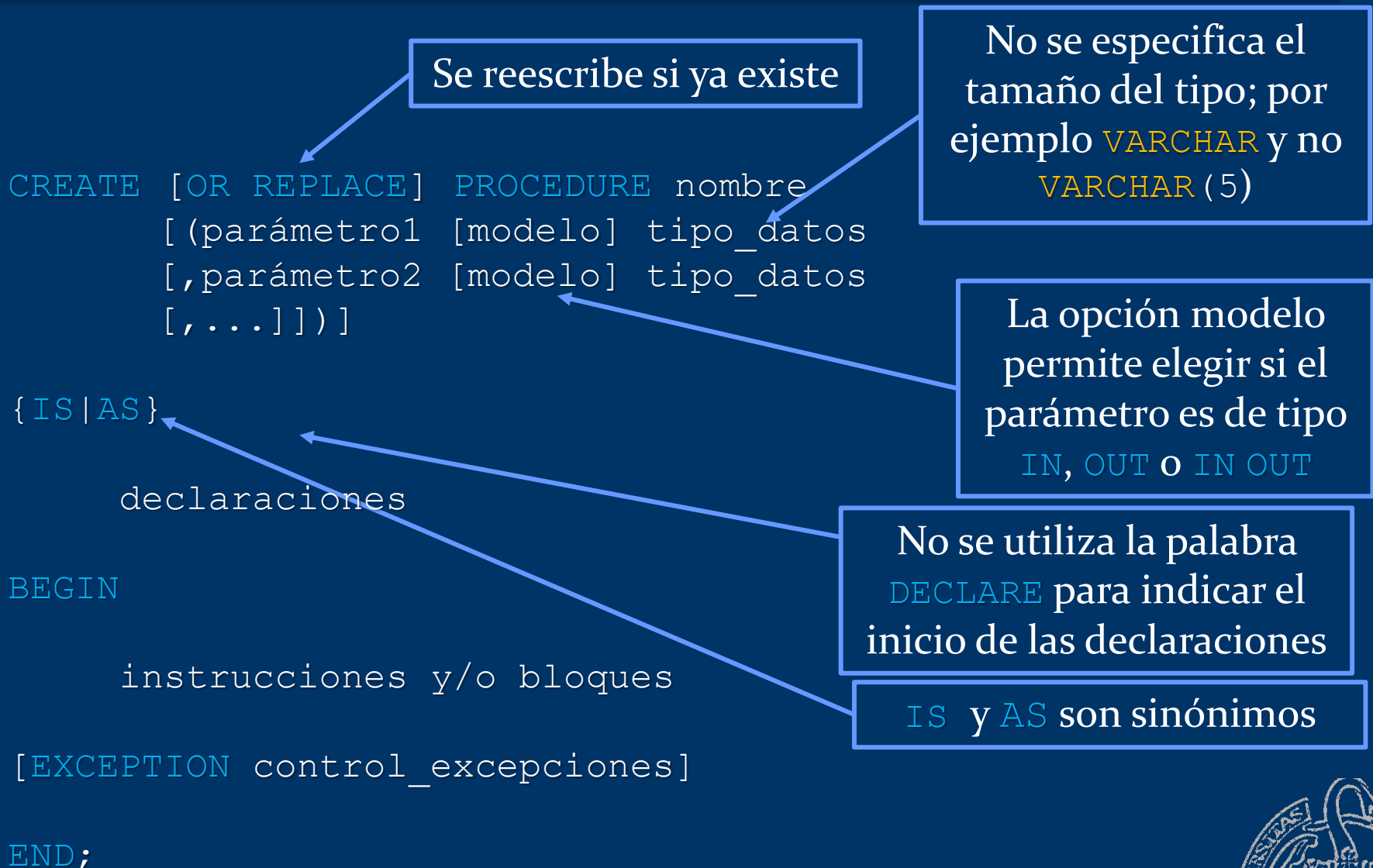


Procedimientos

- ✓ Un procedimiento es el mecanismo de Oracle para almacenar bloques de código PL/SQL de forma persistente
- ✓ Reciben un nombre (y quizás argumentos), lo que permite ejecutarlos a partir de ese nombre
 - Los bloques tal y como los habíamos visto hasta ahora se conocen como bloques anónimos (sin nombre)
- ✓ Se compilan y se almacenan en la base de datos
- ✓ Pueden ser modificados una vez creados (con la instrucción `CREATE OR REPLACE`)
- ✓ Como en otros lenguaje de programación, permiten la reutilización eficiente de código



Procedimientos. Sintaxis



Procedimiento. Parámetros

- ✓ Tipos de parámetros:
 - **IN**: El procedimiento recibe una copia del valor o de una variable ya declarada que se utiliza como parámetro al llamar al procedimiento. No puede modificarlo
 - **IN OUT**: Variable ya declarada cuyo valor puede ser leído y modificado por el procedimiento
 - **OUT**: Requiere una variable ya declarada y la puede modificar. Puede leer el valor antes de la primera modificación, pero leerá **NULL**
- ✓ Si no se indica modo alguno, el predeterminado es **IN**



Procedimiento. Ejemplo (I)

```
CREATE OR REPLACE PROCEDURE pr_registrar_cliente
(p_DNI NUMBER, p_nombre VARCHAR2,
 p_apellidos VARCHAR2, p_fecha DATE)

IS

BEGIN
    INSERT INTO clientes(id, name, apellidos, fecha_afiliación)
    VALUES (p_DNI, p_nombre, p_apellidos, p_fecha);

    DBMS_OUTPUT.PUT_LINE('Insertado cliente: ' || DNI || ' '
                          || NOMBRE || ' ' || APELLIDOS );

END;
```

Invocación del procedimiento (no es necesario un bloque para probar desde la consola):

```
CALL pr_registrar_cliente('123Z', 'Alonso', 'Azuaga',
DATE '2022-11-03');
```



Procedimiento. Ejemplo (II)

```
CREATE OR REPLACE PROCEDURE pr_buscar_cliente
( p_Nombre IN VARCHAR2, p_DNI OUT VARCHAR2)
IS
BEGIN
    SELECT dni INTO p_DNI
    FROM clientes
    WHERE nombre LIKE '%' || p_Nombre || '%';

EXCEPTION
    WHEN NO_DATA_FOUND THEN
        DBMS_OUTPUT.PUT_LINE('No se encontró el cliente');
    WHEN TOO_MANY_ROWS THEN
        DBMS_OUTPUT.PUT_LINE('Hay más de un cliente');
END;
```

Invocación del
procedimiento

```
DECLARE
    v_dni clientes.dni%TYPE;
BEGIN
    pr_buscar_cliente('Alonso', v_dni);
    DBMS_OUTPUT.PUT_LINE(v_dni);
END;
```



Funciones

- ✓ Son bloques de código PL/SQL con nombre que además devuelven un resultado
- ✓ Sintaxis:

```
CREATE [OR REPLACE] FUNCTION nombre_función
    [(parámetro1 [modelo] tipo_datos
    [,parámetro2 [modelo] tipo_datos [...]])]
RETURN tipo_datos
{IS|AS}
    declaraciones
BEGIN
    instrucciones...
    RETURN expresión; -- Puede haber varios RETURN
                        -- en distintos lugares
    [EXCEPTION control_excepciones]
END;
```



Funciones. Ejemplo (I)

```
CREATE OR REPLACE
  FUNCTION fn_ahorro_IVA (x NUMBER)
  RETURN NUMBER
IS
BEGIN
  RETURN x/1.21;
END;
```

*¡Llegan las rebajas!
¡Ahórrate el IVA!*

Ejemplo de invocación de la función:

```
BEGIN
  DBMS_OUTPUT.PUT_LINE(fn_ahorro_IVA (100));
  DBMS_OUTPUT.PUT_LINE(100-0.21*100);
END;
```

```
82,64  -- Esto es lo que pago realmente
79      -- Esto es lo que creía que iba a pagar
```



Funciones. Ejemplo (II)

- ✓ Función que devuelve el peso medio de todos los productos almacenados en una tabla:

```
CREATE OR REPLACE FUNCTION fn_peso_medio RETURN NUMBER
IS
v_peso productos.peso%TYPE;

BEGIN
    SELECT AVG(peso)
    INTO v_peso
    FROM productos;

    RETURN v_peso ;
END;
```

- ✓ Esta función se podría invocar desde una instrucción SQL:

```
SELECT * FROM productos
WHERE peso > fn_peso_medio;
```

¿Por qué querías hacer esto?



Bloques dentro de procedimientos y funciones

- ✓ Un bloque se puede entender como una única instrucción, por lo que se pueden incluir en el cuerpo de un procedimiento (o función)
- ✓ Pueden tener incluidas declaraciones (`DECLARE`) cuyo ámbito será el del propio bloque, y no se extenderán al resto del procedimiento
- ✓ También pueden incluir, como cualquier bloque, su propia sección de tratamiento de excepciones, por lo que se puede implementar el típico control de excepciones del tipo try-catch
- ✓ Se puede redefinir una declaración, suplantándola y recuperándola al finalizar el bloque



Bloques dentro de procedimientos y funciones

```
DECLARE
    v_N  VARCHAR(10);
BEGIN
    v_N := 'A';
    DECLARE
        v_N  FLOAT;
    BEGIN
        v_N := 1.4142;
        DBMS_OUTPUT.PUT_LINE(v_N);
    END;
    DBMS_OUTPUT.PUT_LINE(v_N);
END;
```

```
Statement processed.
1.4142
A
```



Acerca de *Creative Commons*

Licencia CC (*Creative Commons*)

Este tipo de licencias ofrecen algunos derechos a terceras personas bajo ciertas condiciones.

Este documento tiene establecidas las siguientes:

- ① Reconocimiento (*Attribution*):
En cualquier explotación de la obra autorizada por la licencia hará falta reconocer la autoría.
- ② No comercial (*Non commercial*):
La explotación de la obra queda limitada a usos no comerciales.
- ③ Compartir igual (*Share alike*):
La explotación autorizada incluye la creación de obras derivadas siempre que mantengan la misma licencia al ser divulgadas.

Material original elaborado por Iván Martínez y Fernando Sáenz, con modificaciones de Javier Arroyo, Yolanda García y Virginia Francisco

